Computational Vision

Laboratory Guide

Faculty: Prof. Alexandra Moutinho
Eng. Alexandre Athayde

February 2026

Introduction

The present document has the objective to aid the students in the laboratory component of the curricular unit **Computational Vision**, describing the objectives and required tasks to develop at each laboratory session, which aim to apply key-concepts and algorithms of Computer Vision (CV).
Following CV's syllabus, this guide is organized into two different parts, each of them approaching separate topics of this course. The first one, *Image Processing*, presents some fundamental ideas about computational vision using a single camera and the common techniques for processing of images. This first part is then complemented by providing comprehensive insight about feature detection after the processing stage, exploring multiple techniques. In the second part, *Computer Vision*, an introduction of using deep learning for feature recognition is made, and the fundamentals of projective geometry and camera models are introduced. These are used to explore the topic of stereo vision, which is particularly useful in distance measuring and 3D reconstruction.
Each laboratory session will provide certain skills that are useful in understanding the theoretical basis of CV, and, furthermore, these are expected to help in the development of the CV projects.

- Reading materials:
  The laboratories require some level of preparation, and therefore reading materials are provided to the students using the **CV partners webpage**, and will be updated as the classes progress. These materials are the **Theoretical Classes slides** and the respective sections of the **bibliographic references**, and, in specific classes, will be complemented with other materials. Particularly for the laboratory sessions, two courses offered by MathWorks will be helpful in Part 1, **Introduction to Image Processing** (link) and **Image Segmentation, Filtering, and Region Analysis** (link), and can be audited for free from the Coursera platform.
  Since the implementation will be made using MATLAB, a set of recommended MATLAB functions (denoted in a specific font: `function`) and/or Simulink blocks (represented in italic: *Simulink Block*) is provided in the guide for each laboratory session.

- Evaluation:
  At the end of each laboratory session, a quiz will take place, with a small number of questions to be answered directly about the topics addressed in the session. These questions can be relative to the respective lab and about theoretical concepts, so adequate preparation is advised. Note that, during the implementation during the laboratory sessions, you will work in groups of 3 students, but the quiz at the end of each session is individual.

Final Note: the recommended version of MATLAB is **2023a**, with the respective Simulink version, so please be sure to install it. If you are unable to do so, you may use the computers available in the laboratory. The required toolboxes for MATLAB are the ***Image Processing Toolbox***, ***Computer Vision Toolbox***, ***DSP System Toolbox***, ***Statistics and Machine Learning Toolbox***, and the ***Deep Learning Toolbox***, and, if cameras are to be used, the necessary drivers for the operational system and the ***Image Acquisition Toolbox Support Package for OS Generic Video Interface*** add-on should be installed as well.

Contents

I Image Processing **2**

1 Colour Spaces and Histograms **3**

2 Spatial Filters **7**

3 Point Processing and Binary Morphology **11**

Part I

Image Processing

Chapter 1

Colour Spaces and Histograms

Following the first-week lectures, the respective laboratory session addresses the representation of digital images, namely different colour spaces and how these are related, and provides some initial insight about the histogram of grayscale images.

Objectives

- Familiarization with image loading and visualization tools in MATLAB and Simulink
- Experiment different colour spaces
- Learn about the histogram of a grayscale image

MATLAB and Simulink functions

- Image loading: `imread`, *From Video Device*, *Image From File*
- Image visualization: `imshow`, `montage`, *To Video Display*
- Colour space conversion: `rgb2gray`, `hsv2gray`, `rgb2hsv`, `im2gray`, *Colour Space Conversion*
- Histogram visualization: `imhist`, *2-D Histogram and Array Plot*

Tasks

1. Load and visualize an image in MATLAB or Simulink

Expected time of completion for task: 10 minutes.

Naturally, the first step in image processing is to load an image. The image to be loaded should be a coloured one since it will be useful to explore different colour spaces. Do not forget that the image file should be in the working directory of MATLAB, or the path to it should be added.
The image used as an example is provided in the file *bottlecaps.jpg*, and is the following:

Figure 1.1: RGB example image.

2. Experiment using different colour spaces

Expected time of completion for task: 20 minutes.

After this, try converting this image into different colour spaces, namely HSV, YCbCr and grayscale. If you are using MATLAB, the `montage` command is useful to visualize multiple images. Alternatively, in Simulink, different *To Video Display* blocks can be used. The result should be similar to the following:

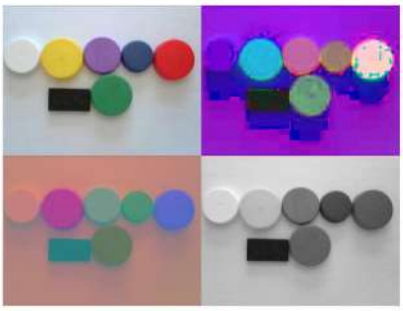

Figure 1.2: Example image in RGB (top left), HSV (top right), YCbCr (bottom left) and grayscale (bottom right).

Do not forget: you are not visualizing the images in different colour spaces, as `imshow`/`montage` interprets them in RGB; you are seeing the interpretation of RGB of a different colour space!

3. Obtain and visualize the histogram of a grayscale image

Expected time of completion for task: 10 minutes.

Lastly, obtain the histogram of a grayscale image and visualize it. As an example, a different image can be used, namely the one provided in the file *autumn.tif*, which is built-in in MATLAB (and therefore you do not need the file itself). Both the original coloured image and the respective grayscale conversion are shown on figure 1.3:

Figure 1.3: RGB and grayscale-conversion of the *autumn.tif* image (credited to MathWorks).

The respective histogram is:

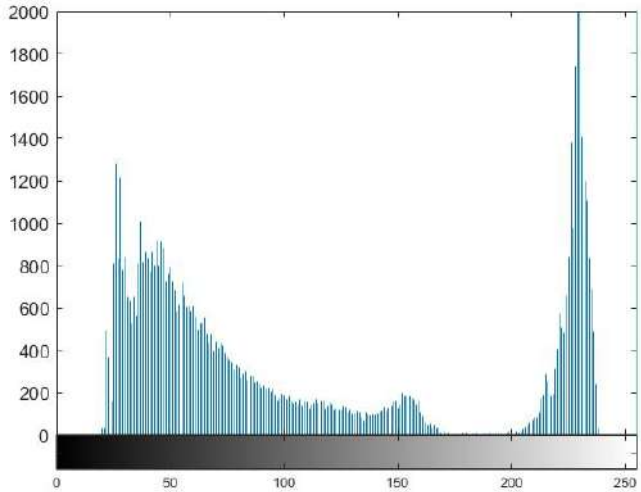

Figure 1.4: Histogram of the grayscale-conversion of the *autumn.tif* image (credited to MathWorks).

Experiment obtaining the histograms of different images and try to draw conclusions about how each of the histograms is related with the original grayscale image. What is expected to be the histogram of a dark image? Or a bright one?

Further work: correctly select a certain colour

Knowing how the different colour spaces work, it is now possible to filter the pixels of an image based on the values they present on a given colour space. For an 8-bit image in the RGB colour space, it can be assumed that a pixel is white if all its three components are above a certain value, which can be, for example, 200. Afterwards, it is possible to design an algorithm that generates a new image from the first one, separating the pixels that fulfil the mentioned condition and the ones that do not. Such procedure is presented on Algorithm 1, and the results shown in figure 1.5. Despite being somewhat more intricate, a similar algorithm can be used in Simulink. Please note that the separation is obviously not perfect, but this topic will be addressed in detail in further theoretical and laboratory classes.

Algorithm 1 Detecting specific colours in an RGB image

```
[a, b, c] = size(RGB)
                                        ▷ Extracts size of the original RGB image.
NewImg = zeros(a, b)
    ▷ Generates a matrix with zeros with the same number of rows and columns of the RGB image.
for i = 1, ..., a do
    for j = 1, ..., b do
        if RGB(i, j, 1) > 200 and RGB(i, j, 2) > 200 and RGB(i, j, 3) > 200 then NewImg(i, j) = 255
                                        ▷ If all components of the pixel are above 200, it assumes it as white.

        else NewImg(i, j) = 0
                                        ▷ If not, it assumes the pixel as black.
        end if
    end for
end for
```

Figure 1.5: Detecting white-coloured pixels in figure 1.1.

Try to use the procedure described above to detect each of the colours of the bottle caps in figure 1.1. Think about which colour space is more adequate, and what conditions should be imposed in the values of the pixels. Please note that the goal here is to provide a general understanding of the colour spaces and how these can be used to extract information from images, and not to detect all the bottle caps without flaw.

Chapter 2

Spatial Filters

Once you know how to work with images using MATLAB and Simulink, it is now time to start to understand the more common image processing techniques. As you may know from the second lecture of this course, one of such techniques is spatial filtering, which has a multitude of different applications, all of which have their importance in image processing. In this second laboratory session, the effects of different image noise type are explored, and how such noise can be removed using spatial filters is shown.

Objectives

- Learn how different noise types affect a grayscale image
- Explore several algorithms in spatial filtering for image smoothing
- Understand the advantages and limitations of particular algorithms

MATLAB and Simulink functions

- Image noise: `imnoise`
- Image filtering: `fspecial`, `imfilter`, `imgaussfilt`, `medfilt2`, `conv2`, *2-D FIR Filter*
- Advanced image filters: `modefilt`, `imnlmfilt`, `imbilatfilt`, `wiener2`

Tasks

Since Simulink itself does not have dedicated blocks for introducing noise in images nor for many of the filters to be used, a MATLAB code implementation is suggested. Nonetheless, after you complete the tasks for this session, you can, as an additional and optional task, implement the same algorithms in Simulink, for which the method is described after Task 3.
By now, you are expected to know how to load images, visualize them, and convert them to a grayscale image in MATLAB (and in Simulink), so these steps are omitted. When in doubt, refer to Laboratory 1.

1. Corrupt an image with different types of noise

Expected time of completion for task: 10 minutes.

Firstly, load one of the provided images and convert it to grayscale. Afterwards, use the aforementioned functions to introduce noise in it, and try to draw conclusions about the differences among noise types. It is useful to read the documentation of the MATLAB functions (**hint**: use the `help` command) so you know how to execute them properly, as well as which parameters they have. Experiment each noise type with different intensities as well.
As an example, we will use the *car1.jpg* image, which is shown below, already converted to an grayscale image and corrupted with different noise types:

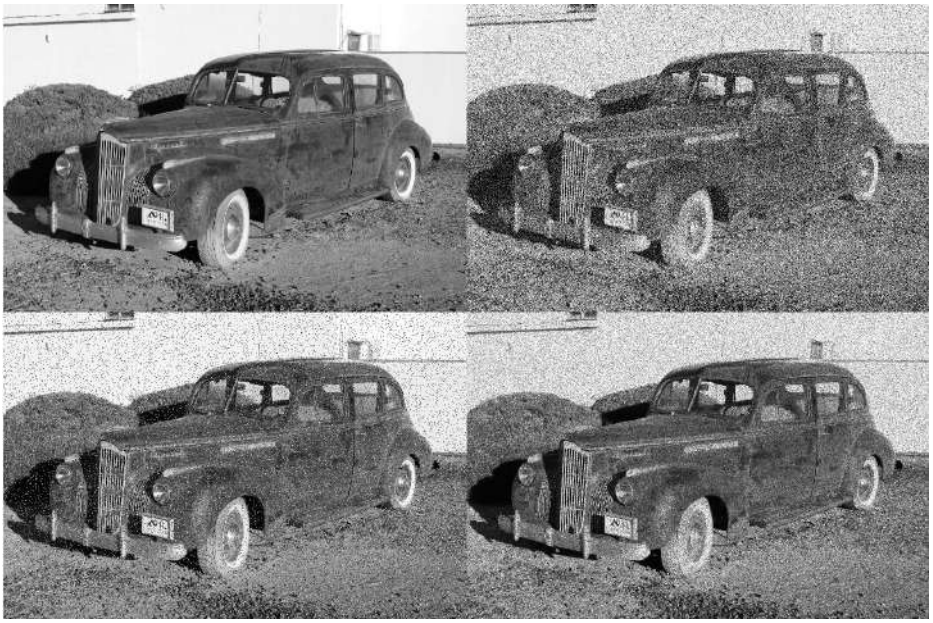

Figure 2.1: Different types of noise on the grayscale image of *car1.jpg* image (credited to MathWorks). The original image is on the top left, followed by its corruption with Gaussian (top right), salt-and-pepper (bottom left) and speckle noise (bottom right), all with 0.1 intensity.

2. Explore different filters

Expected time of completion for task: 20 minutes.

Once the image has been corrupted with noise, you can now use filters to try to suppress such noise, aiming to clean or smooth the image. Once again using the mentioned MATLAB functions, try to remove the noise of the corrupted image. For now, try doing this by applying **averaging**, a **Gaussian** and a **median** filter.
MATLAB allows you to directly filter an image by providing a respective mask, but you can achieve similar results using a convolution of such mask with the image. However, if you choose to use the latter, be aware of the data type it requires (*double*, *single*, *uint8*, ...).
An example of applying a Gaussian filter to an image corrupted with Gaussian noise is shown below:

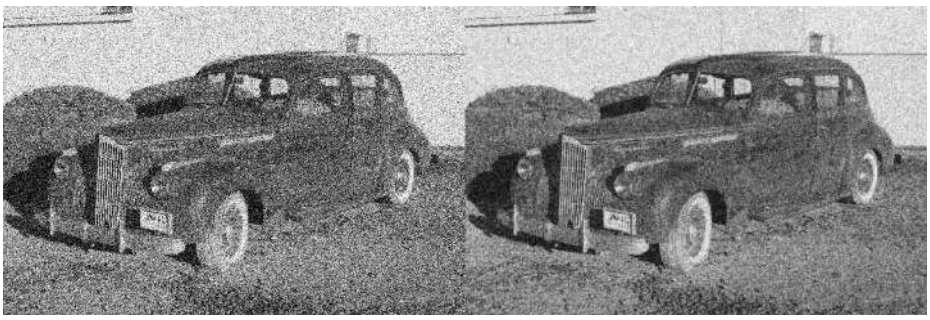

Figure 2.2: Filtering Gaussian noise by using a Gaussian filter.

Strive to determine the advantages of each filter and the respective limitations. Use several intensities for the noise, as well as different mask sizes and parameters for the filters. Does a specific filter work better for one of the noise types? Does it blur the image too much?

3. Removing specific noise types

Expected time of completion for task: 20 minutes.

Now that you have experimented with different noise types and filters, focus on removing Gaussian and salt-and-pepper noises specifically. Initially, use a relatively low intensity for the noise (for example, 0.1) and apply the same filters described in the previous task, and see how they perform when the noise increases. What is or are the best filters for each of these two types of noise?
An example of salt-and-pepper noise removal is shown on figure 2.3.

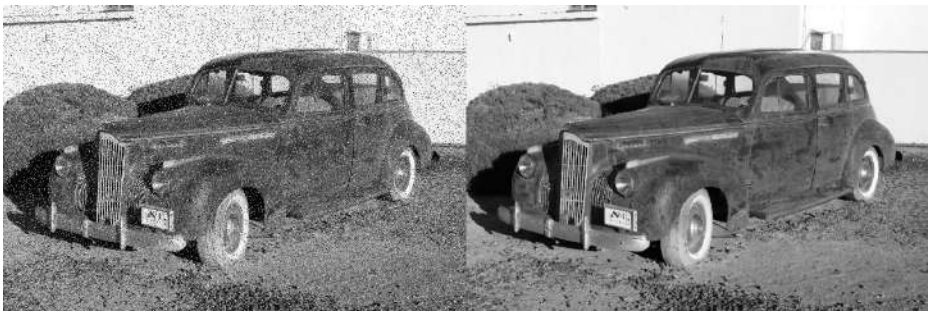

Figure 2.3: Removing salt-and-pepper noise by using an adequate filter.

Once you have experimented with the three filters and already have an idea about how these affect an image, you can work use more advanced ones like the **mode**, **non local means**, **bilateral**, **anisotropic diffusion**, or **Wiener** filters. Can you see improvements regarding the original three filters?

Further work: Simulink implementation

Now that you completed the mandatory tasks, you may wish to implement the same noise-removal algorithm in Simulink. For this purpose, a separate Simulink model has been prepared to facilitate this implementation, and is shown in figure 2.4. In this model, the blocks inside the green and orange areas are MATLAB function blocks that allow for MATLAB functions to be called, where you can implement the noise and filtering options. Read the commentaries inside these blocks for further details. In case of any problems, call for help.

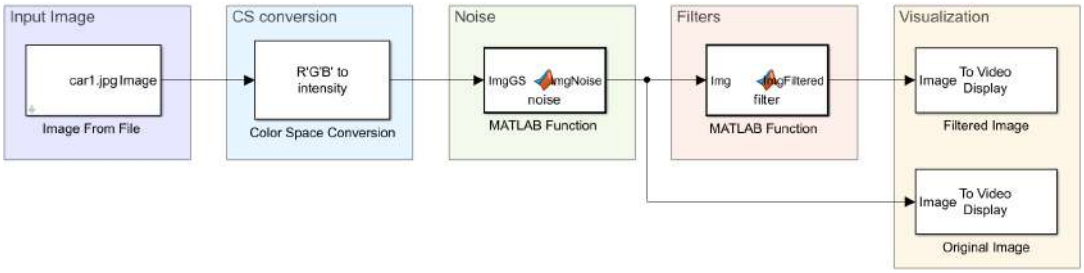

Figure 2.4: Example of a Simulink model for noise removal.

Chapter 3

Point Processing and Binary Morphology

Now that you have had the opportunity to work with grayscale images and learnt how to apply filtering operations for specific goals, some general tools in image processing and segmentation can be studied. In this laboratory session, the common techniques in point processing of images will be addressed, as well as the thresholding methods employed in obtaining binary images. Lastly, several morphological operations will be applied to these last images, which aim to improve the results after thresholding.

Objectives

- Explore the common point processing techniques and study how these impact their histogram
- Learn how to threshold a grayscale image using a suitable method
- Familiarization with different morphological operations

MATLAB and Simulink functions

- Point processing: `imadjust`, `histeq`, *Contrast Adjustment*, *Gamma Correction*, *Histogram Equalization*
- Thresholding: `graythresh`, `imbinarize`, *Compare To Constant*, *Autothreshold*
- Morphological operations: `strel`, `imdilate`, `imerode`, `imopen`, `imclose`, *Dilation*, *Erosion*, *Opening*, *Closing*

Tasks

In the past two sessions, MATLAB code was the chosen environment for implementing the different methods and algorithms studied in the respective theoretical classes. This time however, a Simulink implementation is described in this guide, taking advantage of the fact that it provides specific blocks for the topics of this session. Nonetheless, a MATLAB code implementation is suggested as further work. Similarly to the previous laboratory class, it is assumed that you already know how to obtain grayscale images in both MATLAB and in Simulink. The image to be used is the *cells.png*, shown on figure 3.1:

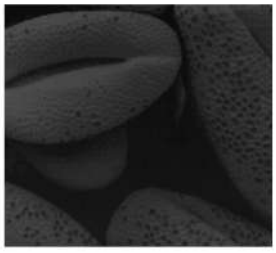

Figure 3.1: Image of *cells.png* image (credited to MathWorks).

1. Point Processing of Grayscale Images

Expected time of completion for task: 20 minutes.

In Simulink, assemble a model that allows you to visualize grayscale images using the recommended blocks, in order to be able to perform point processing operations, and to obtain the respective histogram. An example of a suitable Simulink model is provided below, where the different processing methods are implemented inside a *Subsystem* block:

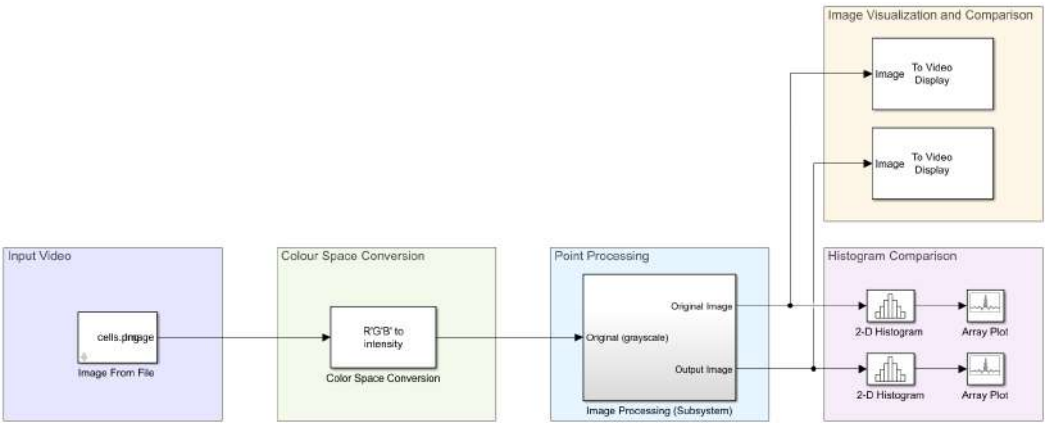

Figure 3.2: Example of a Simulink model for point processing of a grayscale image.

When using the *Image From File* block to load an image, take care to put a **discrete number as the sample time parameter** (0.1 for example), and ensure it is loaded as a **in the *double* type**, since the point processing blocks in Simulink will work for images of this type (and not *uint8*)

With the theoretical background from the third lecture, use different available blocks that allow you to perform **contrast adjustment**, **brightness adjustment**, **gamma correction** and **histogram equalization** on your grayscale image. An example of each of these operations on the original image is shown in figure 3.3, in the same order as described above.

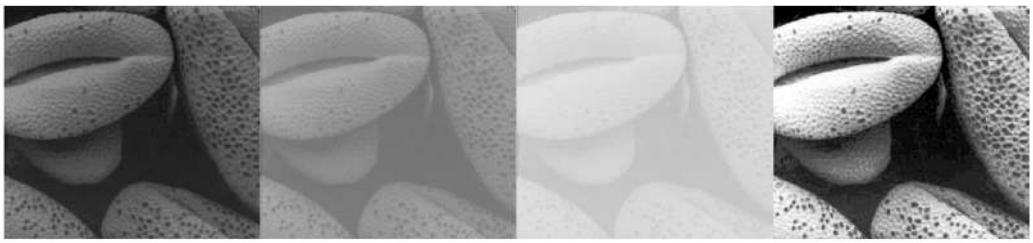

Figure 3.3: Results of the different point processing methods on a grayscale image.

See how these operations impact the respective histogram and provide an adequate explanation for the obtained results for each of them.

2. Binarization of a Grayscale Image

Expected time of completion for task: 10 minutes.

Suppose that your objective was to obtain a binary image in which the pixels that corresponded to a cell (from figure 3.1) were supposed to be white, and the remaining ones were black. Employing an adequate block, obtain a binary image using a **manual threshold**. What is the the value that provides the best results?
Afterwards, try to apply an **automatic threshold** using a suitable block. The resulting image should be similar to the one shown in figure 3.4.

Figure 3.4: Binary image after an automatic threshold method has been applied.

Is it easier to have better results using an automatic method than a manual one? Why? Could the manual method be improved if a point processing technique was used before?

3. Morphological Operations

Expected time of completion for task: 20 minutes.

In this last task, continue your process of obtaining a binary image that adequately identifies the cells in the original picture. However, you can see from figure 3.4 that, even using an automatic threshold, the obtained binary image still has multiple regions in which the pixels corresponding to a cell are not white. Apply an adequate morphological operation to your binary image and try to solve this problem, as shown in figure 3.5. Can you think of other ways, using morphological operations, of reaching a similar result? And what would be the limitations of these operations?

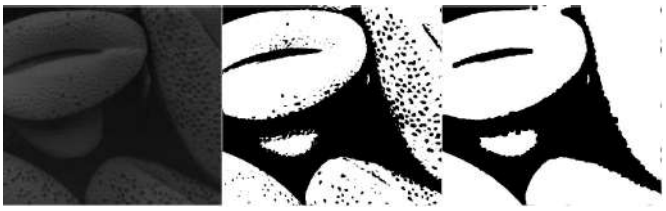

Figure 3.5: Application of morphological operations .

Further Work: MATLAB implementation, adaptive thresholding and additional morphological operations

Regarding the MATLAB code implementation, it has the disadvantage of requiring more steps to perform the same binarization and morphological operations, but it also allows for more operations, more parameters and additional methods to be explored. Start by reading the documentation of certain MATLAB functions, namely the `imadjust`, `imbinarize` and `strel`, using the `help` command. Afterwards, repeat the same three tasks as before, but using MATLAB code instead of a Simulink model.
Lastly, add some *salt-and-pepper* noise to your grayscale image, obtain a binary version of it, and then use morphological operations to **remove the noise** that may appear in this image. Do you need to process your image before you apply the binarization method? Which technique would you use and why? To guide you in this, a suitable method of reaching the described goal of removing the noise of a binary image that adequately thresholds the pixels that correspond to cells is illustrated on 3.6. The order of operations, from the original image with noise to the final binary one, is from left to right. Can you identify each of the operations that was used?

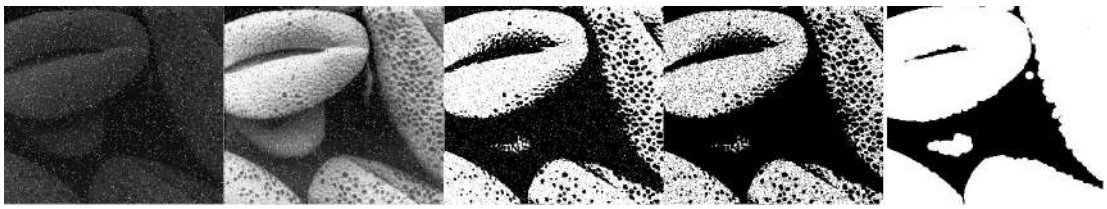

Figure 3.6: Different stages of identifying the pixels that correspond to a cell.

- Adaptive thresholding

  Now, you can progress to some more advanced topics. The first is adaptive thresholding, which is particularly useful in images with varying light conditions. For example, take the *rice.png* (credited to MathWorks) image and its binary version in figure 3.7, after using Otsu's automatic method.

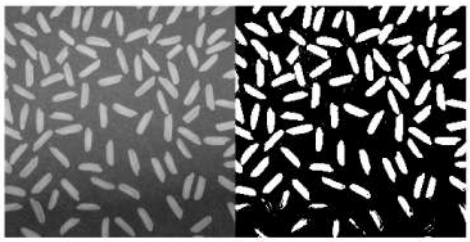

Figure 3.7: Original and binary image of *rice.png* after an automatic threshold.

As you can see, the automatic binarization method, despite providing some very good results in segmenting the rice grains in the top of the image, has a worst performance on its downside. This is due to the fact that the original image has different degrees of light, which leads to a loss of accuracy when using an automatic binarization method, since it employs a global threshold value for the image. A way to counteract this is to use an **adaptive** method that uses the mean value of the intensity of the neighbourhood of the pixel, instead of the whole image. After reading the information MATLAB provides about the `imbinarize` function, try using an adaptive binarization method, and compare it to the original image, like the figure 3.8 illustrates below:

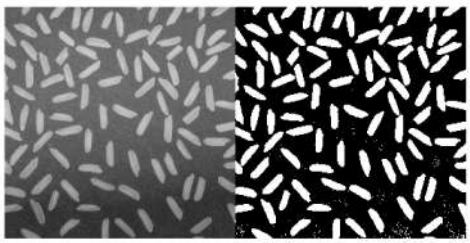

Figure 3.8: Original and binary image of *rice.png* after using an adaptive threshold.

- Additional morphological operations

  In this session, you are supposed to understand the more basic morphological operations, namely *erosion*, *dilation*, *closing* and *opening*. Explore the effect of using **different structuring elements**, with varying sizes, and how these influence the resulting binary image. Beyond these four elementary operations, there are are many other similar ones, for many of which MATLAB provides specific functions, and you can see the comprehensive list of these **here**. Experiment with them and see how they affect the resulting binary image, and try to think about scenarios in which these would be useful.